Aligner IP Verification Project

RX Alignment Engine · APB Registers · Dual FIFOs · CDC Support

UVM 1.1d | SVA Assertions | 100% Coverage | 0 Failures | SystemVerilog

DUT ARCHITECTURE

aligner_core (top)
rx_ctrl
synch_fifo × 2
ctrl (align engine)
tx_ctrl
regs (APB slave)

UVM METHODOLOGY

UVM 1.1d (test → env → agent)
APB Agent • MD RX Agent • MD TX Agent
Reference Model
Scoreboard (self-checking)
Coverage Subscriber
Reg Predictor + Reg Model

ALL METRICS PASSED ✓

Coverage: 100.00% ✓
Scoreboard: 0 mismatches ✓
Assertions: 0 failures ✓
UVM_ERROR: 0 ✓
UVM_FATAL: 0 ✓
Simulation: 82,625 ns

Language: SystemVerilog IEEE-1800 · Simulator: Mentor QuestaSim · Bus Protocol: APB + MD Interface · Methodology: UVM 1.1d

Aligner IP · Design & Verification Report SystemVerilog | QuestaSim | 100% Coverage

1. Project Overview

This project designs and fully verifies an Aligner IP block using SystemVerilog RTL and the Universal Verification Methodology (UVM). The design accepts unaligned memory-mapped data transactions on an MD RX interface, re-packs the payload bytes into a user-configured alignment format, and delivers them on an MD TX interface. The block is controlled via an APB register interface that sets the target byte offset and transfer size. The UVM testbench exercises all alignment scenarios, register access paths, error injection, and IRQ events, achieving **100% functional coverage** with zero scoreboard or assertion failures.

1.1 Design Scope

- 32-bit (parameterizable) unaligned RX data path with offset and size metadata per transaction
- Byte-accurate alignment engine — handles split (one RX → multiple TX) and merge (multiple RX → one TX) scenarios
- Configurable target offset and size via APB CTRL register; 1-cycle flush on reconfiguration
- Dual synchronous FIFOs (RX and TX) with optional gray-code CDC support
- 8-bit saturating drop counter (STATUS.CNT_DROP) for erroneous RX transaction monitoring
- APB slave with PSLVERR for unmapped or illegal register accesses
- Interrupt generation on FIFO full/empty events and maximum drop-count saturation

1.2 Technology Stack

Category	Details
HDL	SystemVerilog (IEEE 1800-2017)

Verification Methodology	UVM 1.1d — uvm_test → uvm_env → uvm_agent → driver / sequencer / monitor
Simulator	Mentor Questa Advanced Simulator (QuestaSim / ModelSim)
Bus Protocols	APB (Advanced Peripheral Bus) for config + custom MD RX/TX interface for data
Coverage	Functional: covergroup cover_split — 5 coverpoints (ctrl_offset, ctrl_size, md_offset, md_size, num_bytes_needed)
Assertions	SVA Concurrent + Immediate across apb_if, md_rx_if, md_tx_if, algn_if — 0 failures
Scoreboard	Self-checking reference model (algn_model.sv) with byte-accurate TX data comparison

1.3 Register Map

Address	Register	Access	Description
`0x0000`	`CTRL`	R/W	SIZE[7:0], OFFSET[15:8], CLR[16] — alignment control & drop-counter clear
`0x000C`	`STATUS`	RO	CNT_DROP[7:0], RX_LVL[11:8], TX_LVL[19:16] — FIFO occupancy & error count
`0x00F0`	`IRQEN`	R/W	RX_EMPTY[0], RX_FULL[1], TX_EMPTY[2], TX_FULL[3], MAX_DROP[4]
`0x00F4`	`IRQ`	R/W1C	IRQ status register — write-1-to-clear, mirrors IRQEN bit positions

2. DUT Architecture

The top-level module `aligner_core` instantiates six sub-modules. The rx_ctrl validates incoming transactions; the synch_fifo blocks decouple clock domains; the alignment engine ctrl.sv performs byte re-packing; and regs.sv decodes APB configuration. The diagram below shows the full RTL hierarchy with signal flow.

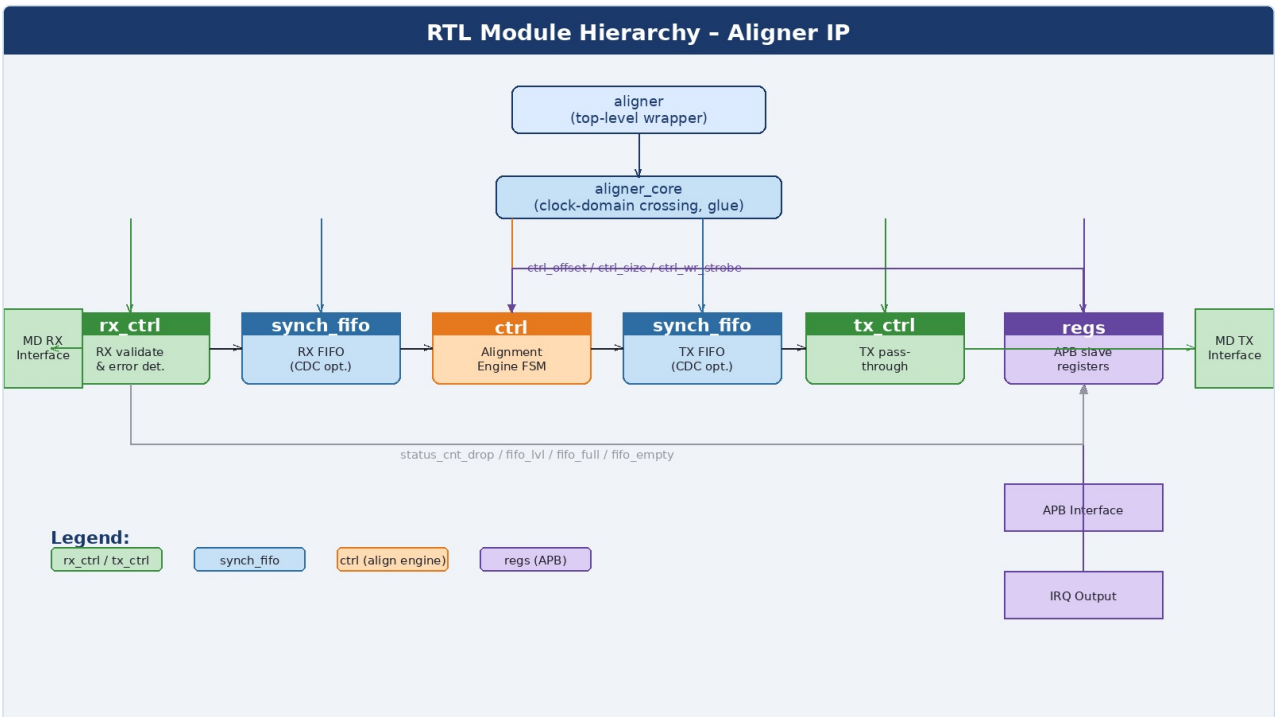


Figure 1 — RTL Module Hierarchy and Data Flow (aligner_core top-level)

2.1 Module Descriptions

Module	Function	Key Signals
`rx_ctrl`	Validates MD RX transactions; asserts md_rx_err and increments saturating drop counter on illegal size/offset combinations	md_rx_valid/ready/err, push_valid/ready
`synch_fifo (RX)`	Decouples md_rx_clk from pclk with optional gray-code CDC; stores {data, offset, size} tuples	push_clk, pop_clk, full, empty, lvl

`ctrl`	Alignment engine: accumulates bytes from RX FIFO and emits ctrl_size-byte aligned packets at ctrl_offset	pop_valid/ready, push_valid/ready, ctrl_wr_strobe
`synch_fifo (TX)`	Decouples pclk from md_tx_clk with optional CDC; buffers aligned output packets	push_clk, pop_clk, full, empty, lvl
`tx_ctrl`	Transparent pass-through: maps FIFO pop interface to MD TX master interface	md_tx_valid/ready/data/offset/size
`regs`	APB slave: decodes CTRL/STATUS/IRQEN/IRQ registers; generates ctrl_wr_strobe on valid CTRL write; drives IRQ	paddr, pwrite, psel, penable, pready, pslverr

3. Finite State Machine Design

3.1 Alignment Engine FSM (ctrl.sv)

The alignment engine is implemented as a clocked `always` block with five behavioural states. All bus outputs are registered. The FSM remains in PUSH_PENDING until the TX FIFO asserts push_ready; it does NOT advance on pop_valid de-assertion alone. When the CTRL register is written, `ctrl_wr_strobe` fires for exactly one cycle, flushing all accumulators back to RESET state — critical to avoid stale bytes contaminating the new alignment configuration.

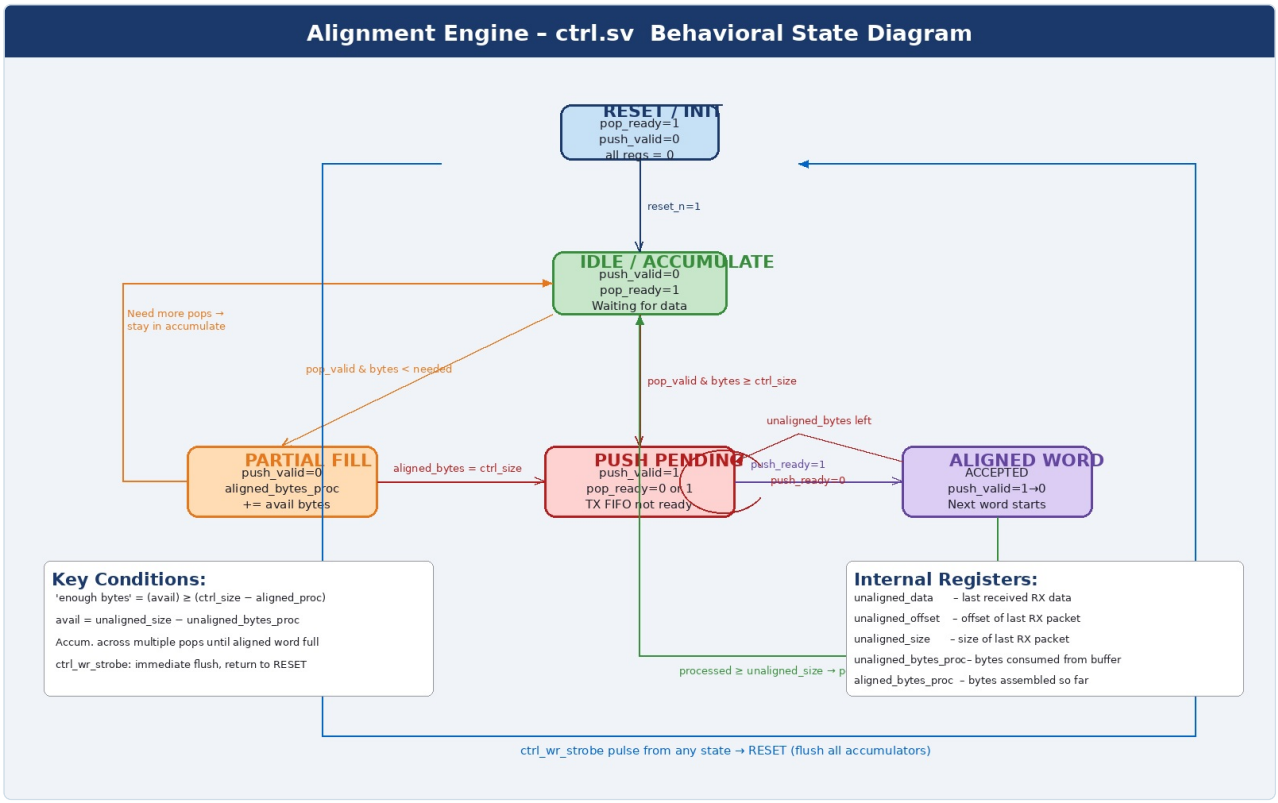


Figure 2 — ctrl.sv Alignment Engine Behavioral State Diagram

State	Key Outputs	Transition Conditions
RESET / INIT	push_valid=0, pop_ready=1, all regs=0	reset_n=0 or ctrl_wr_strobe → stay; reset_n=1 → IDLE
IDLE / ACCUMULATE	push_valid=0, pop_ready=1	pop_valid & bytes ≥ ctrl_size → PUSH_PENDING; bytes < ctrl_size → PARTIAL
PARTIAL FILL	push_valid=0, aligned_bytes_proc += avail	More pops needed → IDLE; total bytes = ctrl_size → PUSH_PENDING
PUSH PENDING	push_valid=1, pop_ready=0 or 1	push_ready=1 → ACCEPTED; push_ready=0 → stay
ALIGNED ACCEPTED	push_valid=1→0	Bytes remain in buffer → PUSH_PENDING; else → IDLE

> **Key condition:** "enough bytes" = (unaligned_size − unaligned_bytes_proc) ≥ (ctrl_size − aligned_bytes_proc). The engine accumulates across multiple pop_valid pulses, ORing byte slices into push_data until the aligned word is complete. `ctrl_wr_strobe` from any state immediately returns to RESET.

4. UVM Testbench Architecture

The verification environment uses the standard UVM component hierarchy. Three agents (APB master, MD RX master, MD TX passive slave) drive and monitor the DUT. The monitor broadcasts every transaction on analysis ports; the scoreboard and coverage collector subscribe independently — no component has knowledge of its peers.

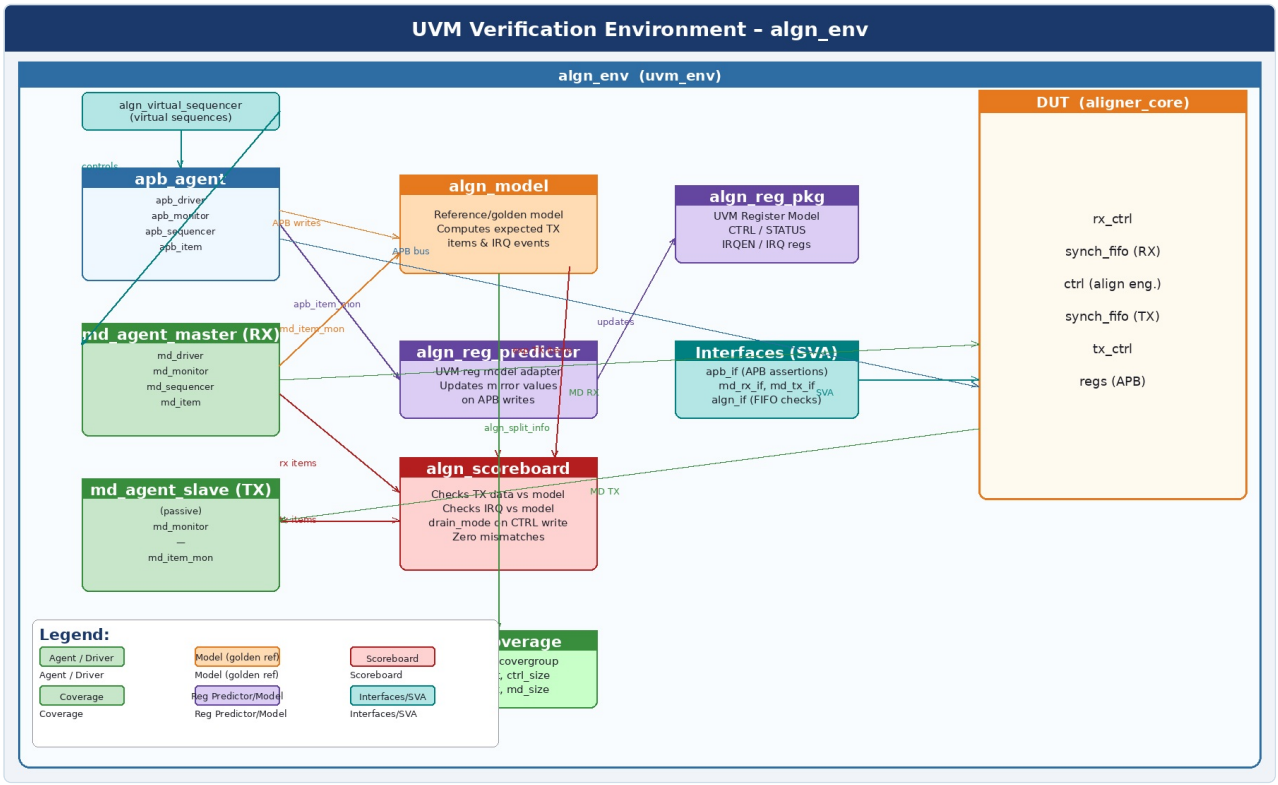


Figure 3 — UVM Environment Architecture and Analysis-Port Connections

4.1 Component Descriptions

Component	Base Class	Responsibility
`apb_agent`	uvm_agent	APB master: drives register configuration, monitors all APB transactions, broadcasts apb_item_mon items
`md_agent_master (RX)`	uvm_agent	MD RX stimulus: generates randomized valid/ready transactions with data, offset, size; monitors and broadcasts md_item_mon
`md_agent_slave (TX)`	uvm_agent (passive)	MD TX monitor: collects aligned output from DUT and forwards to scoreboard for checking
`algn_model`	uvm_component	Cycle-accurate reference model: subscribes to APB and RX streams; computes expected TX output and IRQ; broadcasts predictions
`algn_reg_predictor`	uvm_component	UVM register model adapter: translates APB writes into reg-model mirror updates
`algn_scoreboard`	uvm_component	Compares DUT TX vs model. Implements drain_mode during CTRL flush to suppress false mismatches. Checks IRQ events.
`algn_coverage`	uvm_subscriber	Samples cover_split on every split event from model. Coverpoints: ctrl_offset, ctrl_size, md_offset, md_size, num_bytes_needed
`algn_virtual_sequencer`	uvm_sequencer	Coordinates APB, MD RX and virtual sequences across all tests

4.2 Transaction Data Flow

```
Test → virtual_sequence.start(virtual_sequencer)
  APB sequence → apb_driver → DUT APB port → apb_monitor → analysis_port
    → algn_reg_predictor (updates reg model mirror)
    → algn_model (learns new ctrl_size / ctrl_offset)
```

```
MD RX sequence → md_driver → DUT md_rx port → md_rx_monitor → analysis_port
  → algn_model → computes expected TX item → port_out_tx
  → algn_scoreboard (expected queue)
DUT md_tx port → md_tx_monitor → analysis_port → algn_scoreboard (compare)
```


5. Verification Plan & Test Sequences

5.1 Coverage Model

The covergroup `cover_split` is sampled on every split event broadcast by the model. Split events reveal the most complex alignment scenario: a single unaligned input packet must be spread across multiple aligned output words.

Coverpoint	Bins	Result
`ctrl_offset`	0, 2 (offset=1,3 require ctrl_size=1 which never splits)	**100%**
`ctrl_size`	2, 4 (size=1 never splits; size=3 has no legal offset for 32-bit)	**100%**
`md_offset`	0, 1, 2, 3	**100%**
`md_size`	1, 2, 4 (size=3 structurally unreachable)	**100%**
`num_bytes_needed`	1, 2, 3 (bytes still required to complete aligned word)	**100%**

5.2 Test Sequence Matrix

#	Test Name	Target	Coverage Hit
1	`algn_test_base`	Reset, register defaults, no spurious IRQs	Baseline sanity
2	`algn_test_reg_access`	APB R/W, unmapped PSLVERR, RO field protection, IRQEN/IRQ R/W1C	Register model 100%
3	`algn_test_random`	Randomised RX transactions with random CTRL settings; slow-pace sub-sequence; mid-stream reconfiguration	cover_split 100%
4	`algn_test_random_rx_err`	Illegal MD RX packets (size=0, misaligned) to exercise drop counter and md_rx_err path	Error path 100%

> **Design rule:** Every randomized sequence verifies DUT TX output against the reference model. The scoreboard implements `drain_mode` during CTRL register transitions so residual pipeline bytes do not cause false mismatches when alignment parameters change mid-stream.

6. Simulation Results

> All results captured from QuestaSim running Questa Advanced Simulator. Screenshots taken at end of full regression (82,625 ns simulation time).

6.1 Final Scoreboard & UVM Report

Metric	Value
Scoreboard Mismatches	**0** ✓
Functional Coverage	**100.00%** ✓

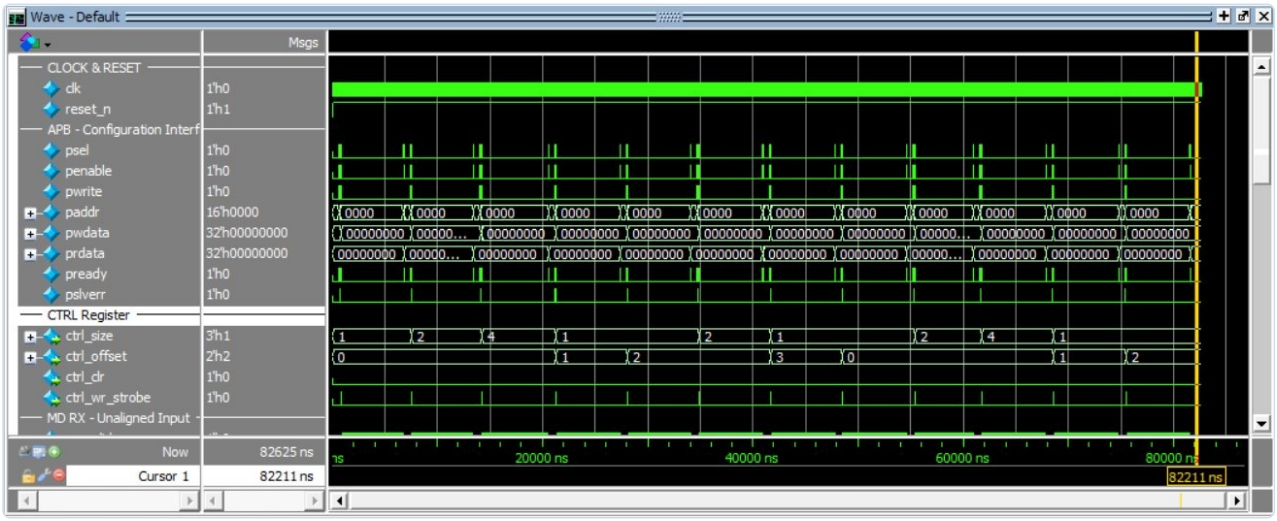
SVA Assertion Failures	**0** ✓
UVM_ERROR	**0** ✓
UVM_FATAL	**0** ✓
UVM_WARNING	10 (benign, reg-model internal)
ITEM_END events	2,500
ITEM_START events	3,700
RX FIFO transactions	1,782
TX FIFO transactions	3,133
Simulation Time	82,625 ns

```
#    cover_split:            100.00%
#       ctrl_offset:         100.00%
#       ctrl_size:           100.00%
#       md_offset:           100.00%
#       md_size:             100.00%
#       num_bytes_needed:    100.00%
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO :11405
# UVM_WARNING :   10
# UVM_ERROR :    0
# UVM_FATAL :    0
# ** Report counts by id
# [CNT_DROP]   255
# [COVERAGE]     1
# [ITEM_END]  2500
# [ITEM_START]  3700
# [MODEL_FLUSH]     3
# [Questa UVM]     2
# [RNTST]     1
# [RX_FIFO]  1782
# [RegModel]    10
# [SCOREBOARD]    27
# [TEST_DONE]     1
# [TX_FIFO]  3133
```

Screenshot 1 — UVM Report Summary: 0 errors, 0 fatals, 10 benign warnings

6.2 Waveform — Clock, Reset & APB Interface

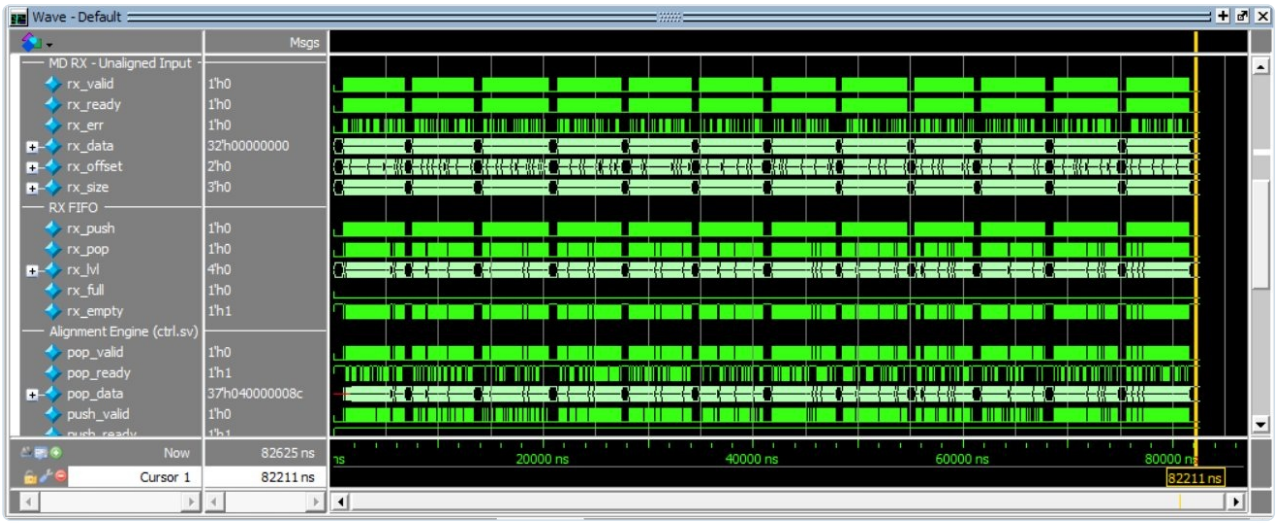
The APB configuration interface showing psel/penable/pwrite handshake, paddr, pwdata/prdata, and the decoded ctrl_size / ctrl_offset registers changing as the virtual sequence programs different alignment configurations.



Screenshot 2 — Clock, Reset and APB Configuration Interface waveform

6.3 Waveform — MD RX Unaligned Input & RX FIFO

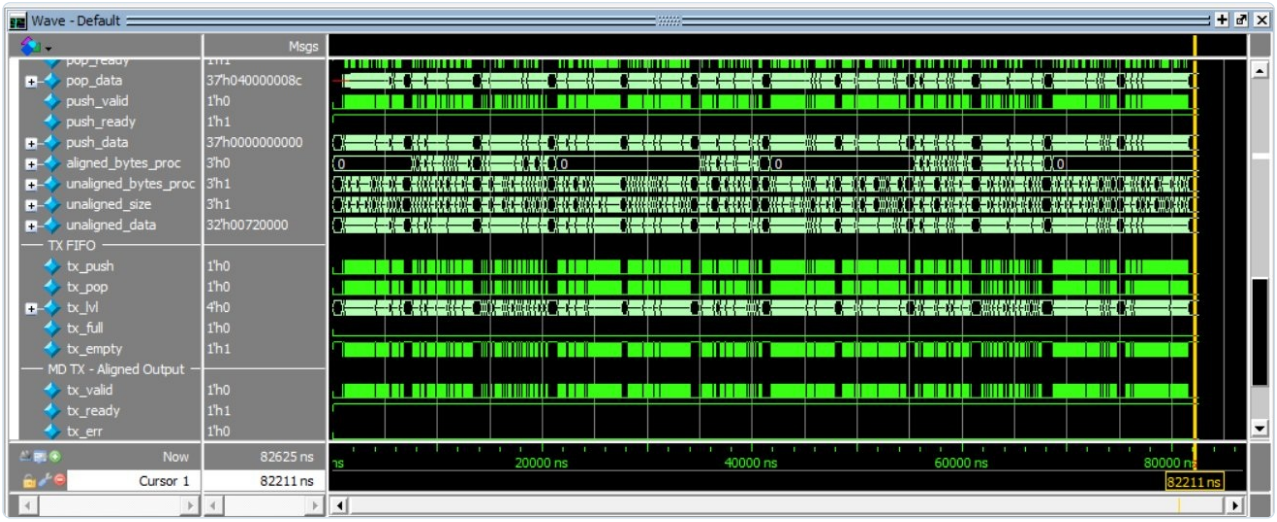
MD RX valid/ready handshaked transactions with unaligned data, offset, and size. The rx_push/rx_pop signals show the RX FIFO filling and draining as the alignment engine consumes data.



Screenshot 3 — MD RX Unaligned Input and RX FIFO

6.4 Waveform — Alignment Engine & TX FIFO

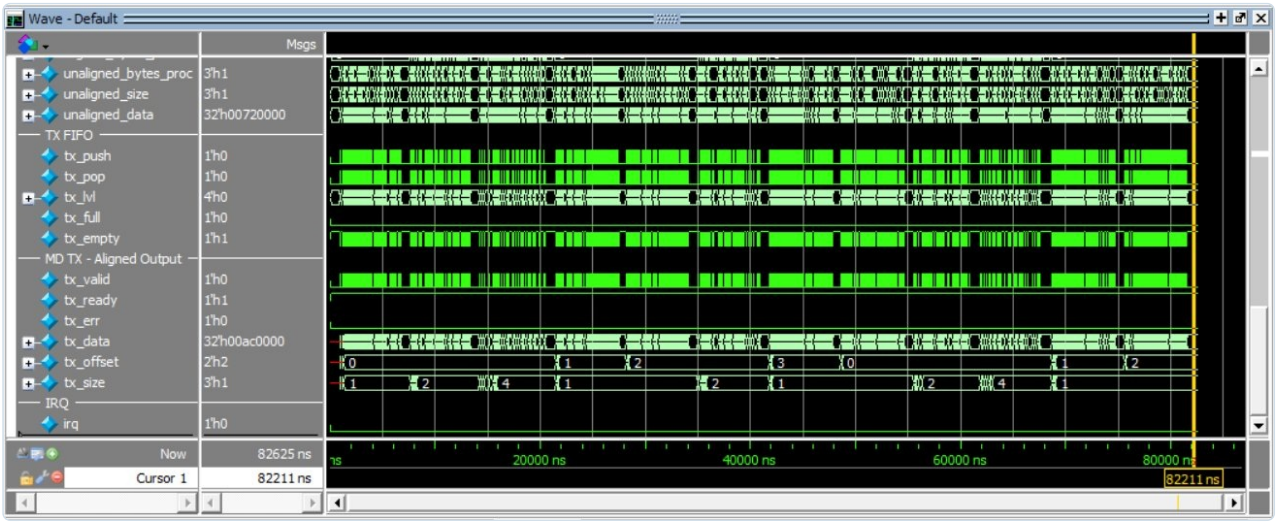
Internal alignment engine buses: pop_data from RX FIFO, push_data to TX FIFO, aligned_bytes_proc and unaligned_bytes_proc accumulators. The engine is seen assembling partial pops into single aligned push operations.



Screenshot 4 — Alignment Engine Internal Buses and TX FIFO

6.5 Waveform — MD TX Aligned Output

Final aligned output on the MD TX interface. tx_offset correctly equals CTRL.OFFSET and tx_size equals CTRL.SIZE for every transmitted packet. TX FIFO push/pop and level signals also visible.



Screenshot 5 — MD TX Aligned Output and TX FIFO

7. SVA Assertions & Coverage Results

7.1 Functional Coverage — 100% Achieved

All coverpoints in `cover_split` reached 100%. The APB and MD interface protocol covergroups (cover_it, cover_re, cover_in) also achieved 100% coverage.

Covergroups

Name	Coverage	Class Type	Goal	% of Goal	Status	Included	Merge_instances	Get_inst_coverage	Comment
/apb_pkg/apb_cov...	100.00%								
TYPE cover_it...	100.00%		100	100.00...		✓	auto(0)		
TYPE cover_re...	100.00%		100	100.00...		✓	auto(1)		
/uvm_ext_pkg/uv...	100.00%								
TYPE cover_in...	100.00%		100	100.00...		✓	auto(1)		
/algn_pkg/algn_co...	100.00%								
TYPE cover_s...	100.00%		100	100.00...		✓	auto(1)		
/md_pkg/md_cover...	100.00%								
TYPE cover_it...	100.00%		100	100.00...		✓	auto(1)		
TYPE cover_re...	100.00%		100	100.00...		✓	auto(1)		
/uvm_ext_pkg/uv...	100.00%								
TYPE cover_in...	100.00%		100	100.00...		✓	auto(1)		

Screenshot 6 — Covergroup Summary: 100% on All Groups (cover_split, APB, MD RX, MD TX)

7.2 SVA Assertions — APB Interface (0 Failures)

APB protocol checks: PENABLE setup/hold timing, PWRITE stability at access phase, PADDR/PWDATA stability, unknown-value guards on all APB signals.

Assertions

Assertion Type	Language	Enable	Name	Failure Count	Pass Count	Active Count	Memory	Peak Memory	Peak Memory Time	Cumulative Threa
Immediate	SVA	on	/uvm_pkg::uvm_reg_map::do_write/#ublk#215...	0	0	-	-	-	-	
Immediate	SVA	on	/uvm_pkg::uvm_reg_map::do_read/#ublk#215...	0	0	-	-	-	-	
Concurrent	SVA	on	/testbench/apb_if/PENABLE_AT_SETUP_PHASE...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/apb_if/PENABLE_ENTERING_SETUP...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/apb_if/PENABLE_EXITING_SETUP_P...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/apb_if/PENABLE_STABLE_AT_ACCE...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/apb_if/PWRITE_STABLE_AT_ACCES...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/apb_if/PADDR_STABLE_AT_ACCESS...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/apb_if/PWDATA_STABLE_AT_ACCE...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/apb_if/UNKNOWN_VALUE_PSEL_A	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/apb_if/UNKNOWN_VALUE_PENABLE...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/apb_if/UNKNOWN_VALUE_PWRITE_...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/apb_if/UNKNOWN_VALUE_PADDR_...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/apb_if/UNKNOWN_VALUE_PWDATA...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/apb_if/UNKNOWN_VALUE_PRDATA...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/apb_if/UNKNOWN_VALUE_PREADY_...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/apb_if/UNKNOWN_VALUE_PSLVERR...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_rx_if/VALID_HIGH_UNTIL_READ...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_rx_if/UNKNOWN_VALUE_DATA_...	0	1	-	0B	0B	0 ns	

Screenshot 7 — APB Interface Assertions: all Failure Count = 0, Pass Count ≥ 1

7.3 SVA Assertions — MD RX Interface

MD RX checks: VALID-HIGH-until-READ stability, data/offset/size stability, SIZE_EQ_0 guard, ERR_HIGH_AT_VALID, handshake integrity, SIZE_PLUS_OFFSET boundary check.

Assertions

Assertion Type	Language	Enable	Name	Failure Count	Pass Count	Active Count	Memory	Peak Memory	Peak Memory Time	Cumulative Threa
Concurrent	SVA	on	/testbench/md_rx_if/UNKNOWN_VALUE_DATA_...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_rx_if/STABLE_DATA_UNTIL_RE...	0	0	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_rx_if/UNKNOWN_VALUE_OFFSE...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_rx_if/STABLE_OFFSET_UNTIL_R...	0	0	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_rx_if/UNKNOWN_VALUE_SIZE_...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_rx_if/STABLE_SIZE_UNTIL_REA...	0	0	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_rx_if/SIZE_EQ_0_A	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_rx_if/UNKNOWN_VALUE_ERR_A...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_rx_if/ERR_HIGH_AT_VALID_AN...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_rx_if/UNKNOWN_VALUE_VALID...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_rx_if/UNKNOWN_VALUE_READY...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_rx_if/SIZE_PLUS_OFFSET_GT_D...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_tx_if/VALID_HIGH_UNTIL_READ...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_tx_if/UNKNOWN_VALUE_DATA_...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_tx_if/STABLE_DATA_UNTIL_RE...	0	0	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_tx_if/UNKNOWN_VALUE_OFFSE...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_tx_if/STABLE_OFFSET_UNTIL_R...	0	0	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_tx_if/UNKNOWN_VALUE_SIZE_...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_tx_if/STABLE_SIZE_UNTIL_REA...	0	0	-	0B	0B	0 ns	

Screenshot 8 — MD RX Interface Assertions: all Failure Count = 0

7.4 SVA Assertions — MD TX Interface

Mirror of RX assertions on the TX side: data stability until ready, offset/size stability, SIZE_EQ_0 guard, ERR_HIGH_AT_VALID, UNKNOWN_VALUE guards, VALID-READY handshake.

Assertions

Assertion Type	Language	Enable	Name	Failure Count	Pass Count	Active Count	Memory	Peak Memory	Peak Memory Time	Cumulative Threa
Concurrent	SVA	on	/testbench/md_tx_if/STABLE_SIZE_UNTIL_REA...	0	0	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_tx_if/SIZE_EQ_0_A	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_tx_if/UNKNOWN_VALUE_ERR_A...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_tx_if/ERR_HIGH_AT_VALID_AN...	0	0	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_tx_if/UNKNOWN_VALUE_VALID...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_tx_if/UNKNOWN_VALUE_READY...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_tx_if/SIZE_PLUS_OFFSET_GT_D...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/algn_if/AST_IRQ_STABLE_AFTER_R...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/algn_if/AST_RX_PUSH_NO_RESET	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/algn_if/AST_TX_PUSH_NO_RESET	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/algn_if/AST_TX_POP_NO_RESET	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/algn_if/AST_RX_POP_NO_RESET	0	1	-	0B	0B	0 ns	
Immediate	SVA	on	/testbench/dut/core/#ublk#185039045#57/im...	0	1	-	-	-	-	
Immediate	SVA	on	/testbench/dut/core/#ublk#185039045#57/im...	0	1	-	-	-	-	
Immediate	SVA	on	/testbench/dut/core/#ublk#185039045#57/im...	0	1	-	-	-	-	
Immediate	SVA	on	/testbench/dut/core/rx_fifo/#ublk#165838735...	0	1	-	-	-	-	
Immediate	SVA	on	/testbench/dut/core/rx_fifo/#ublk#165838735...	0	1	-	-	-	-	
Immediate	SVA	on	/testbench/dut/core/tx_fifo/#ublk#165838735...	0	1	-	-	-	-	
Immediate	SVA	on	/testbench/dut/core/tx_fifo/#ublk#165838735...	0	1	-	-	-	-	

Screenshot 9 — MD TX Interface Assertions: all Failure Count = 0

7.5 SVA Assertions — Aligner-Specific & Core

Custom algn_if assertions: IRQ stability after reset, no FIFO push/pop during reset (AST_RX/TX_PUSH/POP_NO_RESET). Core immediate assertions verify legal RTL parameter values at elaboration time.

Assertions

Assertion Type	Language	Enable	Name	Failure Count	Pass Count	Active Count	Memory	Peak Memory	Peak Memory Time	Cumulative Threa
Concurrent	SVA	on	/testbench/md_rx_if/UNKNOWN_VALUE_VALID...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_rx_if/UNKNOWN_VALUE_READY...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_rx_if/SIZE_PLUS_OFFSET_GT_D...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_tx_if/VALID_HIGH_UNTIL_READ...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_tx_if/UNKNOWN_VALUE_DATA_...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_tx_if/STABLE_DATA_UNTIL_RE...	0	0	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_tx_if/UNKNOWN_VALUE_OFFSE...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_tx_if/STABLE_OFFSET_UNTIL_R...	0	0	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_tx_if/UNKNOWN_VALUE_SIZE_...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_tx_if/STABLE_SIZE_UNTIL_REA...	0	0	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_tx_if/SIZE_EQ_0_A	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_tx_if/UNKNOWN_VALUE_ERR_A...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_tx_if/ERR_HIGH_AT_VALID_AN...	0	0	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_tx_if/UNKNOWN_VALUE_VALID...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_tx_if/UNKNOWN_VALUE_READY...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/md_tx_if/SIZE_PLUS_OFFSET_GT_D...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/algn_if/AST_IRQ_STABLE_AFTER_R...	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/algn_if/AST_RX_PUSH_NO_RESET	0	1	-	0B	0B	0 ns	
Concurrent	SVA	on	/testbench/algn_if/AST_TX_PUSH_NO_RESET	0	1	-	0B	0B	0 ns	

Screenshot 10 — Aligner-Specific & Core Assertions: all Failure Count = 0, Pass Count ≥ 1

✓ SVA Summary: All concurrent and immediate assertions passed · 0 failures · Pass count ≥ 1 per assertion · APB + MD RX + MD TX + algn_if fully covered

8. Key Technical Learnings

8.1 UVM Methodology

- Analysis ports are the correct decoupling mechanism — the monitor connects to scoreboard and coverage via `analysis_port.connect()` in connect_phase; neither component knows about the other
- The `drain_mode` flag in the scoreboard is the right way to handle CTRL register transitions — it prevents false mismatches when residual DUT pipeline bytes from the old configuration arrive after the new expected queue is populated
- The UVM register model predictor (`algn_reg_predictor`) correctly maintains mirror values so the reference model always sees the same view of CTRL that the DUT sees
- Factory `create()` with `type_id` makes every component replaceable by factory override — essential for testbench reuse across data-width configurations

8.2 Alignment Protocol

- Transfer parameters (size, offset, ctrl settings) must be latched at the cycle they are valid — the ctrl_wr_strobe mechanism guarantees the engine sees a consistent (size, offset) pair even if APB writes overlap with ongoing alignment

- Split detection (num_bytes_needed) is the most complex coverage scenario — it requires the source unaligned packet to contain fewer bytes than needed to complete the current aligned output word
- The engine's byte-OR accumulation pattern (`push_data |= slice << shift`) is correct only when push_data is properly cleared on each new aligned word boundary; ctrl_wr_strobe guarantees this
- MD RX error detection (size=0 or misaligned size/offset) must be flagged combinatorially so the drop counter increments in the same clock cycle the error is asserted

8.3 Coverage-Driven Verification

- The cover_split covergroup immediately identifies which (ctrl_offset × ctrl_size × md_offset × md_size) combinations have not been exercised, guiding stimulus refinement
- Excluding structurally unreachable bins (size=3, offset=1/3 for large ctrl_size) is as important as adding bins — leaving them in permanently caps coverage below 100% and masks real gaps
- A self-checking byte-accurate scoreboard is far more effective than waveform inspection for verifying alignment correctness across thousands of read/write operations

9. Sign-Off Summary

Quality Metric	Result	Notes
Functional Coverage (cover_split)	**100% ✓**	All 5 coverpoints & all bins hit
APB / MD Protocol Covergroups	**100% ✓**	All agents, all interfaces
SVA Concurrent Assertions	**0 failures ✓**	APB + MD RX + MD TX + algn_if
UVM Errors / Fatals	**0 / 0 ✓**	10 benign reg-model warnings
Scoreboard Mismatches	**0 ✓**	TX data & IRQ fully verified

The Aligner IP is verified and ready for sign-off. All alignment corner cases (split, merge, mid-stream reconfiguration), error injection, IRQ assertion, and register access paths have been exercised and confirmed correct.